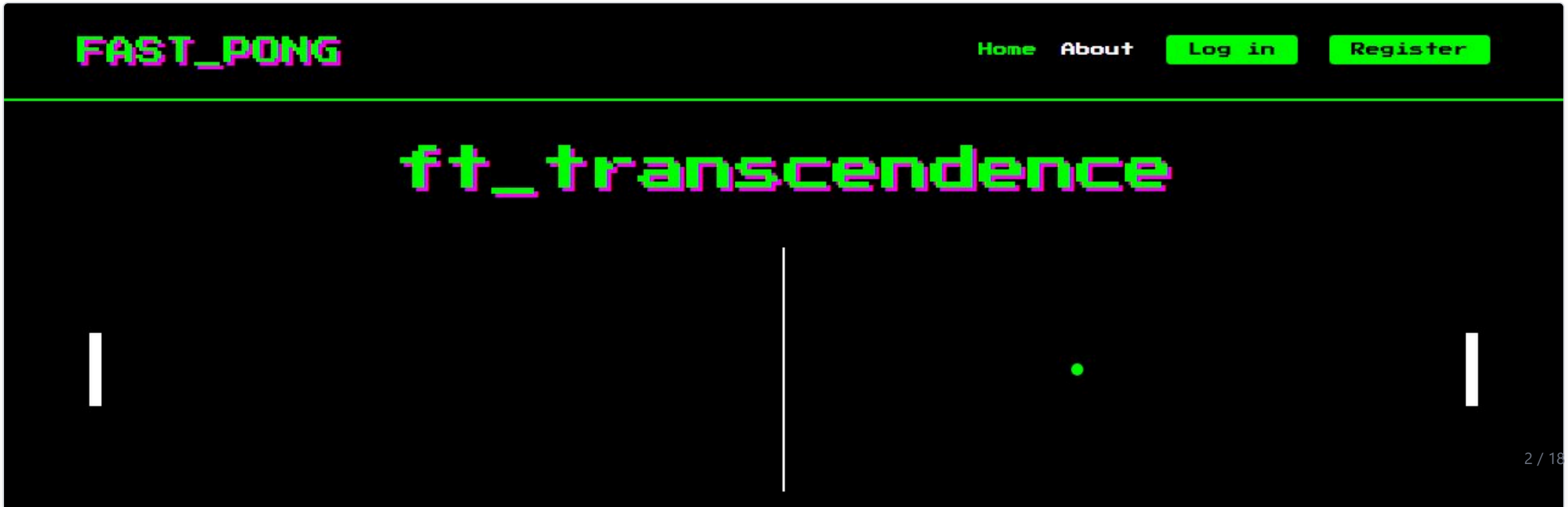
Introduction — what is FAST_PONG?

- Capstone of the 42 common core: a **single-page web app** where people play **Pong in 3D** (Three.js / WebGL) against a friend or an **AI opponent**.
- **Tournaments**: 3–8 nicknames, round-robin matches, automatic tie-breakers, winner announcement.
- **Player stats dashboard**: games played, win-rate pie chart, best score, match history, friends list.
- **Secure access**: email/password or **42 OAuth** (remote authentication), optional **email 2FA**, **JWT** tokens.
- **GDPR**: download my data, anonymize, delete, automatic inactivity cleanup.
- Served over **HTTPS** by Gunicorn in Docker, **PostgreSQL** database, responsive on phones. Bonus feature: a TicTacToe mini-game.



Selected modules (1/2)

Category	Module	Type	How we implemented it
Web	Use a framework as backend (Django)	**Major**	Django 4.2 + DRF; apps `userapp`, `gameapp`, `tournaments`; Gunicorn TLS on :443
	Use a front-end framework or toolkit (Bootstrap)	**Minor**	Bootstrap 4.5 grid/buttons + custom retro CSS, vanilla-JS SPA router
	Use a database for the backend (PostgreSQL)	**Minor**	`postgres:13` container, psycopg2, migrations, named volume
User Management	Standard user mgmt, authentication, users across tournaments	**Major**	Custom `User` (email login, avatar, display name, friends), profile & settings APIs, match history per user
	Implementing a remote authentication (42 OAuth)	**Major**	`redirect_uri` → api.intra.42.fr authorize → SPA `/oauth/callback` → `get-token` code exchange → `/v2/me` → user created/linked → JWT
AI-Algo	Introduce an AI opponent	**Major**	`PongAI`: samples the ball once per second, predicts the intercept point, tunable accuracy / mistake chance / max speed — no A*
	User and game stats dashboards	**Minor**	Profile stat cards + SVG win-rate pie + recent matches, JSON data export with statistics, tournament scoreboard

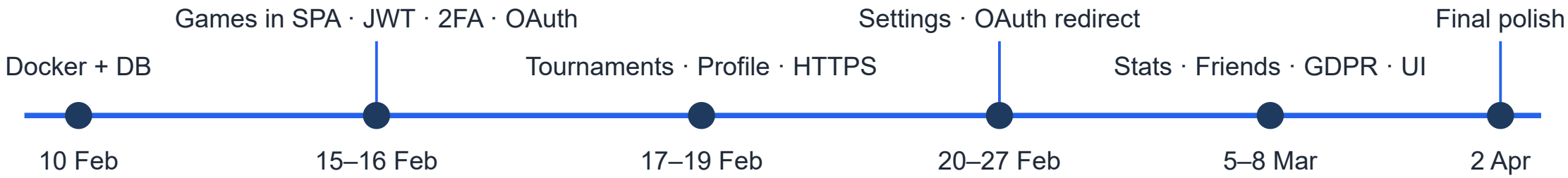
Selected modules (2/2)

Category	Module	Type	How we implemented it
Cybersecurity	GDPR compliance: user anonymization, local data management, account deletion	**Minor**	Export JSON, **Anonymize**, Delete, `delete_inactive_users` command (6 months)
	Two-Factor Authentication (2FA) and JWT	**Major**	Email OTP on login (shared DB cache, async send), SimpleJWT access/refresh tokens
Graphics	Use of advanced 3D techniques	**Major**	Three.js r128: PerspectiveCamera, Phong materials, spot/ambient lights, canvas-generated ball texture, spin physics
Accessibility	Support on all devices	**Minor**	Media queries 1100 / 920 / 768 / 480 px, hamburger menu, fluid game canvas
	Expanding browser compatibility	**Minor**	Standard ES2017 + ES modules, WebGL, tested Chrome & Firefox
	Server-Side Rendering (SSR) integration	**Minor**	Django renders `index.html` (CSRF token, static manifest) — SPA then takes over routing

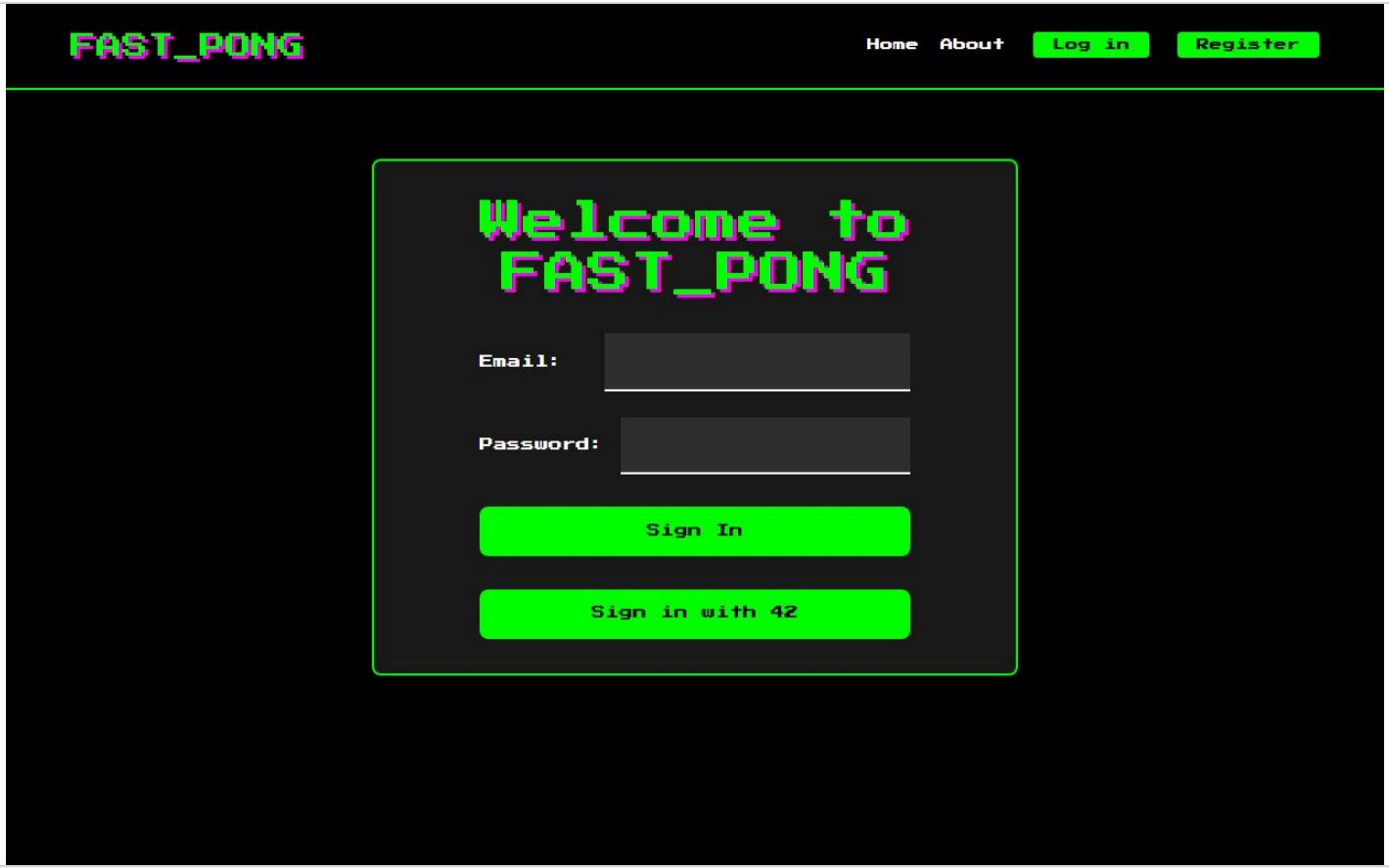
6 Major + **7 Minor** (× 0.5) = **9.5 major-equivalents** · 7 required for 100 %

SDLC model — iterative & incremental

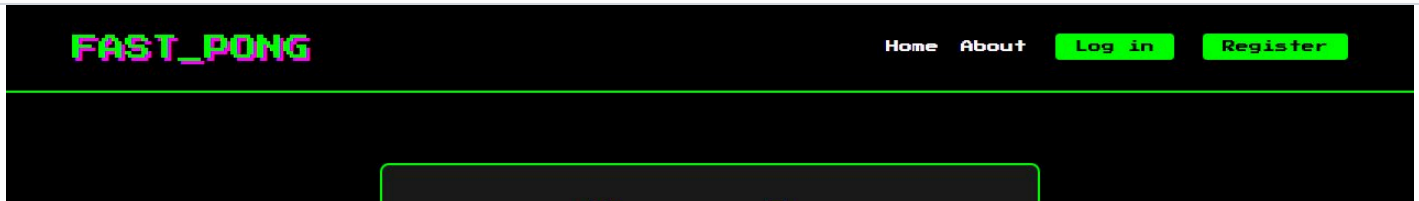
- We worked in **short vertical slices**: build a feature end-to-end → merge → fix what integration broke → next feature.
- **GitHub-flow** branching: 15 pull requests merged in ~4 weeks (`db-connect` , `game-setup` , `tournaments` , `profile-page` , `secure-cookies` , `OAuth` , `user-settings` , `delete-account` …).
- Repeated “Merge branch master into <feature>” commits = continuous integration of parallel work.
- 86 commits, 10 Feb → 2 Apr 2025; bug-fix commits directly follow feature merges (e.g. *registration fix*, *2fa fixed*, *tournament fixed*).
- **Honest note:** no formal sprints, tickets or written specs are in the repo — planning happened in person; requirements came from the 42 subject & module list.



Features — landing, authentication, 2FA

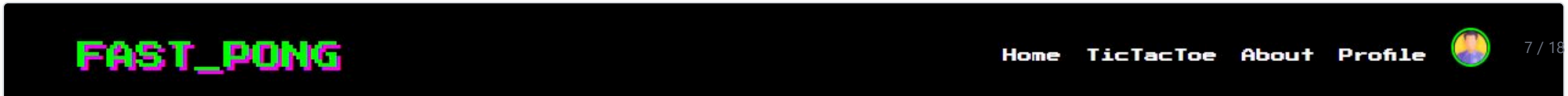


Email / password login or “Sign in with 42” (remote authentication)



Features — stats dashboard & settings (User Management, AI-Algo, GDPR)

Stats dashboard: games played, win-rate pie chart, best score, match history, friends panel



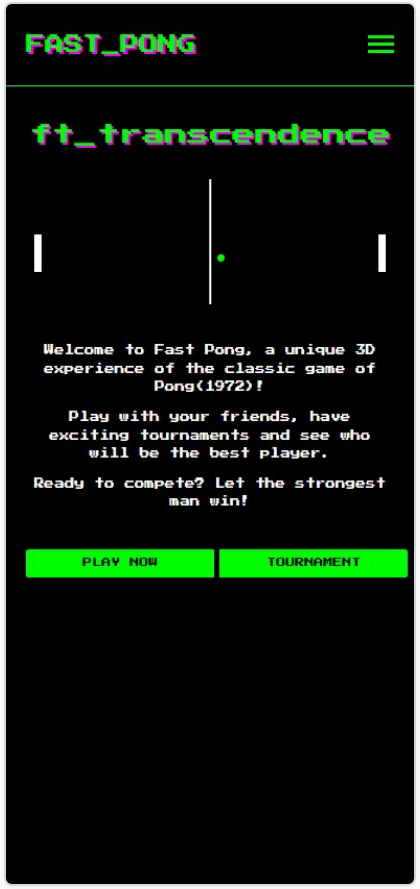
Features — 3D Pong & AI opponent (Graphics, AI-Algo)

Mode selection: Player vs Player / Player vs AI

Features — tournaments & bonus mini-game

Tournament: 3–8 nicknames → round-robin table → Start Match launches Pong → tie-breakers → winner (users across tournaments)

Features — support on all devices (Accessibility)



390 px — home page stacks vertically

Contribution of each member

Derived from git history (`git shortlog -sne` + per-author file analysis) — **team: adjust before presenting**

Salim Hamzaoui

shamzaou · 65 commits

- SPA shell: `index.html` , `script.js` router, `styles.css` , responsive/hamburger
- Integrated Pong (incl. AI paddle) & TicTacToe into the site, match-history UI, win-rate chart
- Profile / settings / avatar, friends list, GDPR pages & export
- HTTPS on 443, secure cookies, OAuth redirect flow, Docker/entrypoint

Nasser Alzaabi

naalzaab · 6 commits

- JWT issuance (SimpleJWT) and email 2FA flow
- 42 OAuth token exchange (`get_token`) — remote authentication
- Port 443 / Docker settings, match JSON API & statistics saving for both games

Alisher Abdullaev

alabdull + legaliab · 9 commits

- Profile page and user-settings endpoints
- Display name & email editing, related migrations
- Delete-account (GDPR) endpoint

Nour Murat

nurmurat · 6 commits

- `tournaments` app: models, views, URLs, tests
- Round-robin generation & tie-breaker (additional matches) system

Limitations (1/2) — architecture & security

- **42 OAuth flow has no `state` parameter** (callback is not CSRF-protected) and the client key has expired — must be rotated before the demo.
- **Tournaments use nicknames, not accounts**; tournament endpoints are protected only by CSRF, not login.
- **Mixed auth**: profile/settings views use the session cookie; other APIs accept the JWT — JWT lives in `localStorage` (XSS-readable).
- No rate limiting / lockout on login or OTP; no password reset or change.
- **AI difficulty** tuning reads a hard-coded `scoreDiff = 0`, so the AI never adapts to the live score.
- Games are local (PvP on one keyboard or vs AI) — no online multiplayer / WebSockets.
- `SESSION_COOKIE_SECURE=False`; debug `print()` of request headers in logs; avatars served via a custom endpoint because `/media/` is not served when `DEBUG=False`.

Limitations (2/2) — operations & leftovers

- **Gmail app password rejected** (SMTP 534 *WebLoginRequired*) — a new app password is needed for real OTP emails.
- CDN dependencies (Bootstrap, jQuery, Three.js, Google Fonts) → the demo machine needs internet.
- GDPR cron file exists but is **not installed** in the image → run `make gdpr-cleanup` manually.
- Dead code: `check_auth` (wrong signing key), `oauth_callback`, `gameapp` models, `django-otp` installed but unused, `production_settings.py`.
- Backend is one Django process (modular monolith) — fine, microservices is not a claimed module.
- UX relies on `alert()` dialogs; leftover merge-conflict comments in `script.js`.
- Only 19 automated tests (auth/2FA/GDPR/tournaments) — no front-end tests.

Recommended further improvements (ranked)

1. **Rotate the 42 client key and the Gmail app password** (or switch to a transactional email API).
2. **Add the OAuth `state` parameter** (`OAUTH_STATE_SECRET` already exists in `.env`) to protect the callback.
3. **Unify authentication on JWT** (or sessions) and require login on tournament endpoints.
4. **Wire AI difficulty to the live score** (replace the hard-coded `scoreDiff`) and expose difficulty levels.
5. Rate limiting + account lockout on login and OTP verification; store tokens in **httpOnly cookies**.
6. Online multiplayer with Django Channels / WebSockets — `asgi.py` and daphne are already present.
7. Vendor CDN assets locally; add password reset; CI running `make test`; remove dead code.

What we fixed in the pre-evaluation audit (Aug 2026)

- **2FA code "sometimes rejected"** — root cause: OTP kept in per-process `LocMemCache`; Gunicorn runs 3 workers, so `verify-otp` usually hit a worker that never stored it. Fix: shared `DatabaseCache` (`django_cache` table) + reuse the code on re-login + trim/str-normalise input.
- **2FA email "very slow"** — root cause: `send_mail()` ran synchronously inside the login request with no timeout. Fix: background thread, `EMAIL_TIMEOUT=10`, failures logged — login now answers in ~80 ms.
- `make test` was broken (wrong settings module) → fixed; suite grew from 2 failing tests to **19 passing**.
- Added the missing **GDPR anonymization** endpoint + button (required by the GDPR module).
- Token-refresh URL bug in the SPA fixed; stale `staticfiles/` now rebuilt at container start.
- A language switcher was built on a wrong module assumption and **removed again** once the module list was corrected.
- Full docs: `docs/study-guide/`, `docs/audit-report.md`.

Demo cheat-sheet (1/2) — setup & accounts

Before the evaluation

- `make build && make up` → **https://localhost** (accept the self-signed cert)
- `make test` → "Ran 19 tests … OK"
- Pre-create: **demo1** / demo1@example.com (no 2FA) and **demo2fa** / demo2fa@example.com (2FA on) — password `Str0ng!Passw0rd`
- Log in as demo1 and play 2–3 games (vs AI) so the stats dashboard has data
- Open a 2nd browser profile to show friends between demo1 & demo2fa

If Gmail / the 42 key are still down

- In `.env` add `EMAIL_BACKEND=django.core.mail.backends.console.EmailBackend`
- `make down && make up`
- Log in as demo2fa → modal appears → `grep "OTP for login" gunicorn-error.log | tail -1` → enter code
- Explain: same code path, only the transport differs
- 42 OAuth: click "Sign in with 42" → show the authorize URL (client_id + redirect_uri); the login completes once the rotated key is in `.env`

Demo cheat-sheet (2/2) — click-path per module

Module	Click-path / what to show
Django · Bootstrap · PostgreSQL	`make logs`, `make db` → `\dt`; `/admin/` with a superuser (`make shell` → createsuperuser); view-source shows Bootstrap CDN + custom CSS
User management	/register → /login → /profile → /settings (edit display name, avatar) → Find Users → Add friend; TOURNAMENT → nicknames → matches
Remote authentication (42)	/login → "Sign in with 42" → authorize page on api.intra.42.fr → back to `/oauth/callback` → logged in; show `get_token` in `userapp/views.py`
AI opponent	PLAY NOW → Player vs AI → play to 3; open `pong.js` `PongAI`: 1-second sampling, intercept prediction, mistake chance
Stats dashboards	/profile after 2–3 games: stat cards, win-rate pie, match history; Settings → Download My Data (JSON statistics); tournament scoreboard
2FA + JWT	Log in as demo2fa → modal → code → DevTools ▸ Application ▸ localStorage `authToken`; decode the JWT payload
GDPR	/settings → Download My Data → Anonymize (2nd account) → Delete; `make gdpr-cleanup`
3D graphics	PLAY NOW → W/S keys, SPACE pause; point at lighting, spin, camera, canvas texture
All devices · browsers · SSR	DevTools device toolbar 390 px → hamburger; open in Firefox too; view-source of / (server-rendered template + CSRF token)
Extra: TicTacToe	Nav ▸ TicTacToe → finish a game → /profile shows a TICTACTOE row

Thank you

Questions? — see docs/study-guide/quick-drill.md